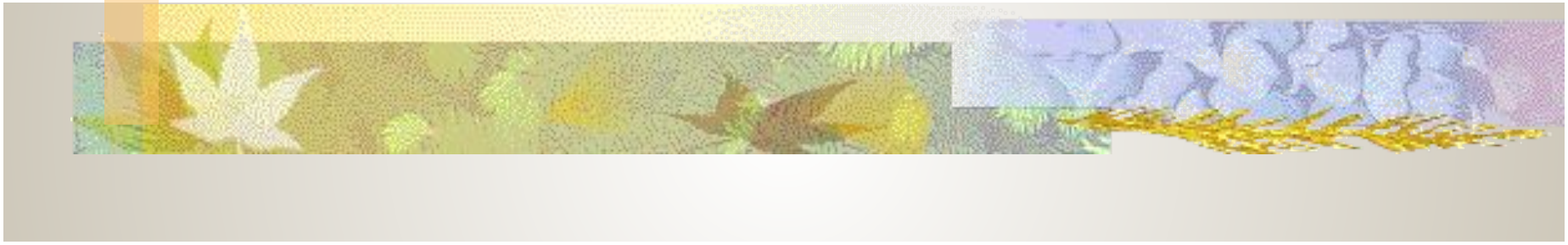
5.Distributed File Systems



Distributed File Systems

- File system provides an abstract view of secondary storage and is responsible for global naming, file access, and overall file organization. These functions are handled by the name service, the file service, and the directory service.
- **File service** is the specification of what the file system offers to its clients.
- **File server** is a process that runs on some machine and helps implement the file service.

File Types

- *Library files*: Generally routines available for use within a user's program. Such files use extensions such as *lib* or *dll*.
- *Program files*: Programs written by users. Such files use extensions that indicate the particular programming language utilized through their extensions, such as *c*, *cpp*, *p*, or *pas*.
- *Object-code files*: Unlinked compiled programs generally in machine language. Such files use extensions such as *o* or *obj*.
- *Compressed files*: Files that have been compressed for storage. Such files use extensions such as Z, zip, or gz.
- *Archive files*: Related files that have been grouped into a single file for storage. Such files use extensions such as *tar* or *arc*.
- *Graphic files*: Binary or ASCII files for printing or viewing. Such files use extensions such as *dvi*, *ps*, *gif*, or *jpeg*.
- *Sound files*: Binary files containing sound data. Such files use extensions such as *midi* or *wav*.
- *Index files*: Index files frequently contain indexing information for other mainframes. Such files use the extension *idx*.
- *Document files*: Files created by a word processor or to be translated by a type-setting program. Such files use extensions such as *doc*, *wp*, *tex*.

Location Transparency

- If the location of a file is communicated, then the name may include the location, machine, and file name, such as myuniversity.edu:/violet/book/chapter8.
- If your distributed system wishes to provide location transparency, then you must provide name transparency through **global naming** just like 1-800 numbers in telephone system.

- A global name space requires the following types of resolution:
- Name resolution-maps human-friendly, symbolic file names to computer file names.
- Location resolution-involves mapping global names to a location. This may be solved by a centralized solution or a distributed solution.

- The centralized solutions create a critical element and a system bottleneck.
- A distributed solution may involve all locations maintaining a complete location resolution table. This approach is not scalable. Therefore, any massive distributed system requires a distributed solution with multiple location resolution servers. Each server is responsible for a particular subset of names. A server location mapping table is consulted to identify what server within the system is responsible for what set of names.

- There are two dominating approaches to segmenting names to the various servers.

1. Provide Hash function to the name

   Server 1 contains names A-B; Server 2 contains names C-D; Server 3 contains names Y-Z;

2. Divide the responsibility based on file types

File Storage

- **Structured files** represent data in terms of records.

Structured file: Record 1

Record 2

Record 3

…

Record N

- Unstructured file: a continuous stream of bytes.

File Attributes

- File name (including file type extension)
- File size
- Type of file ownership (individual or group)
- Name of file owner(s)
- Date of file creation
- Date of last file access
- Date of last modification
- Version number
- Relevant protection information

File Protection Modes

- Read to the file
- Write to the file
- Truncate the file
- Append to the file
- Execute the file
- There are two dominating types of file protection: **access lists** and **capabilities.**

Access lists

- Access list associates with each file a list of users who may access the file and how.
- File 0: (John, *, RWX)
- File 1: (John, staff, R_ _)
- …
- File 3: (*, student, R_ _)

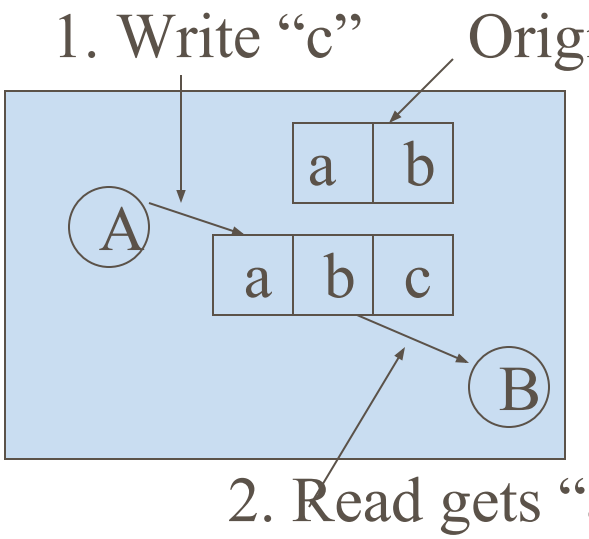
Capability list

- Each user has a kind of ticket, called a capability, for each object to which it has access.
- Process 0

	Type	Rights	Object
0	File	R_ _	Pointer to File 3
1	File	RWX	Pointer to File 4
2	File	RW_	Pointer to File 5
3	Printer	_W_	Pointer to Printer 1

File Modification Notification

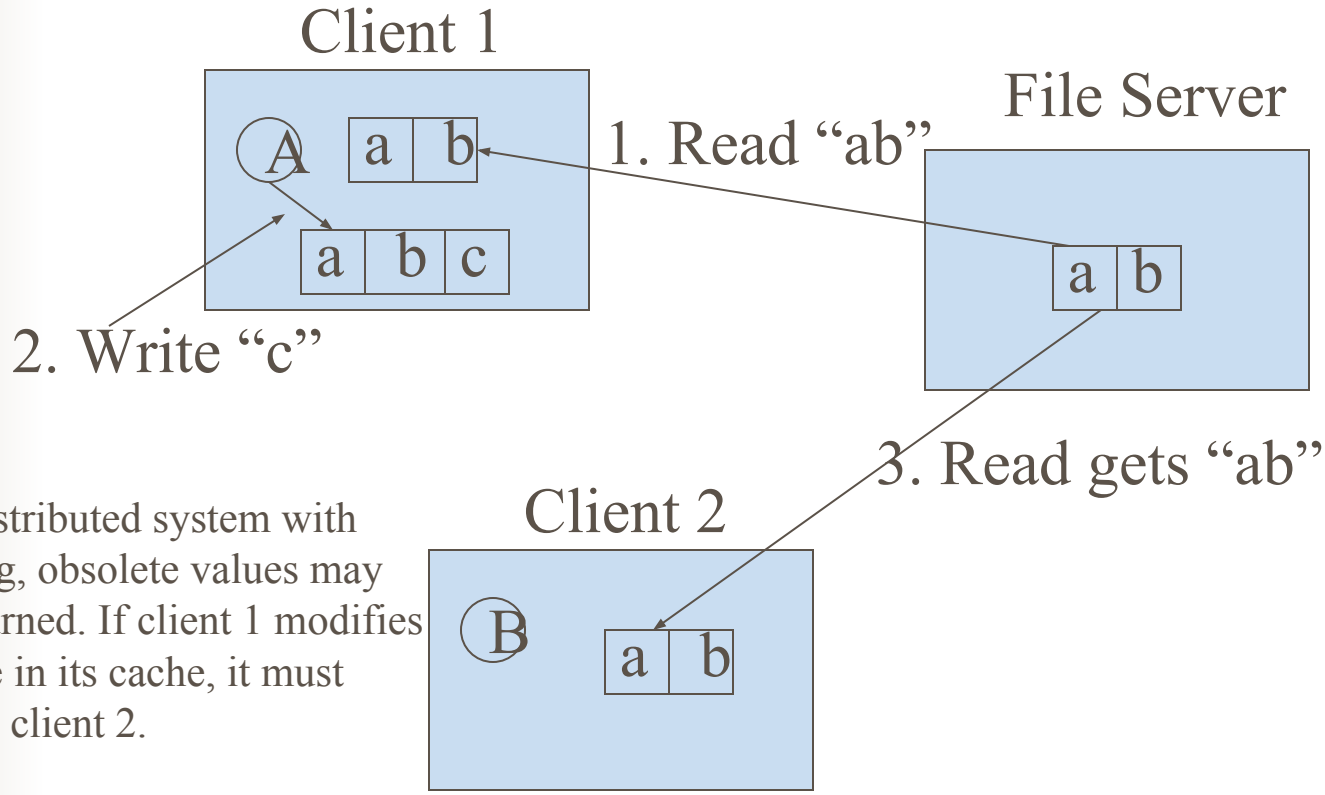
- Single processor



On a single processor, when a READ follows a WRITE, the value returned by the READ is the value just written.

a

- Distributed system



In a distributed system with caching, obsolete values may be returned. If client 1 modifies the file in its cache, it must inform client 2.

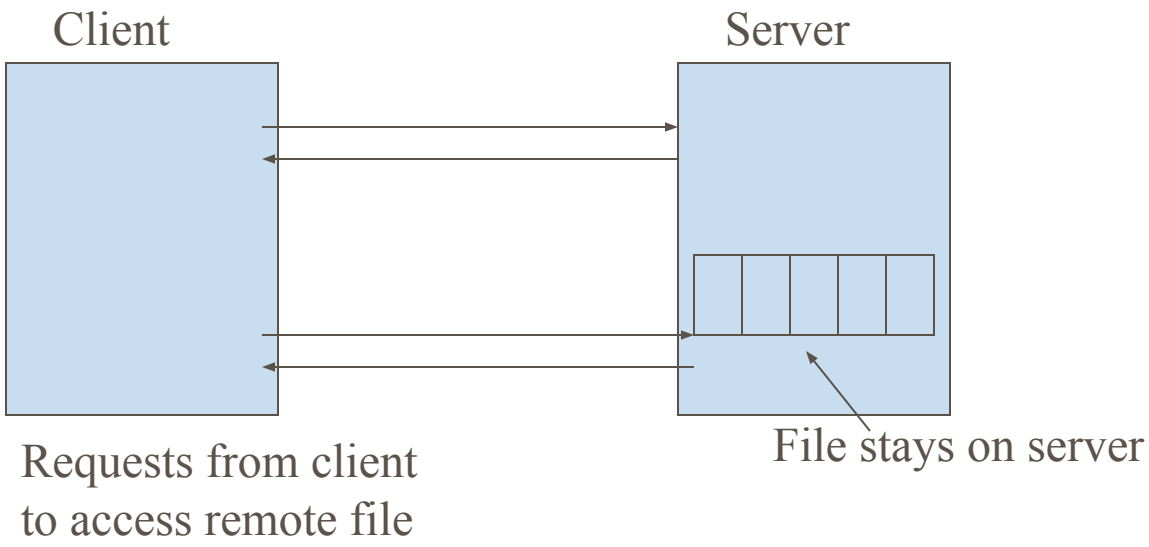
There are two groups of notification methods.

- **Immutable**: modifications are not allowed. With no modification allowed, no damage can occur to the data and no notification needs to take place.
- **Mutable:** or changeable files: three basic approaches.
- *Immediate notification*: with immediate notification, each and every operation to a file is instantaneously visible to every participant holding a copy of the file. This method is very difficult and impractical to implement in a distributed environment
- *Notification on close*: with notification on close, other participants are only notified of file modifications when a participant closes a file and thereby terminates their access to the file.
- *Notification on transaction completion*: A transaction is a fixed set of operation. When this fixed set of operations is completed, members of the system are notified

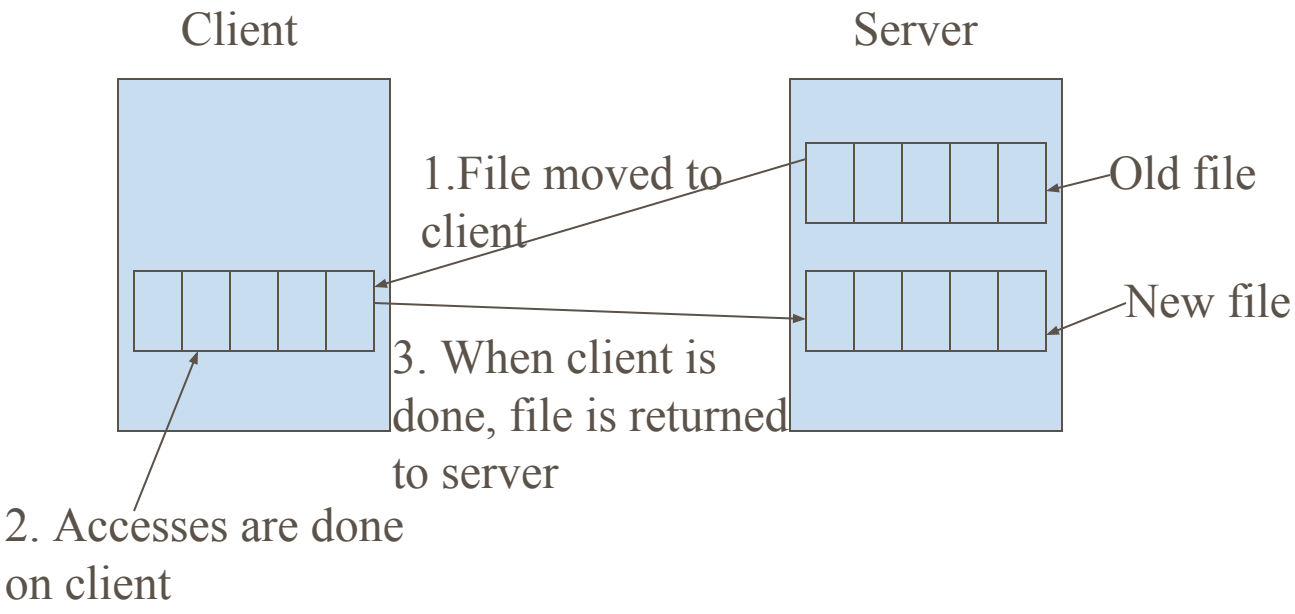
File service implementation

- File service implementations may be based on **remote access** or **remote copy** and may be **stateful** or **stateless**.

Remote access model



Remote copy model

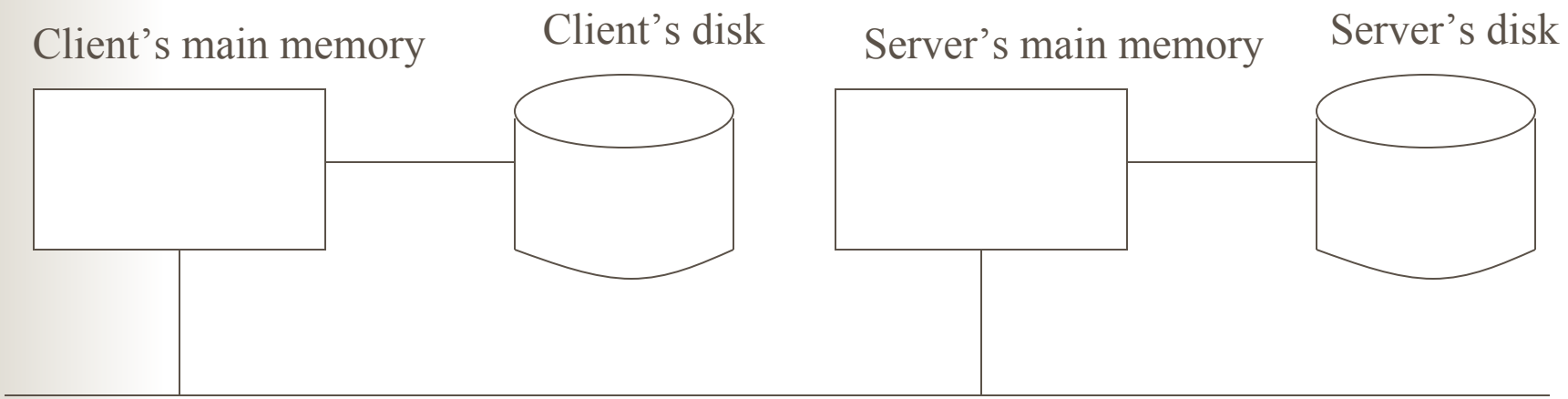


- **A Stateful server** maintains information about all clients that are utilizing the server to access a file.
- **A stateless server** maintains no client information. Each and every request from a client must include very specific request information, such as file name, operation, and exact position in the file. The client maintains the state information.

Advantages of stateful and stateless servers

Advantages of stateless servers	Advantages of stateful servers
Fault tolerance	Shorter request messages
No OPEN/CLOSE calls needed	Better performance
No server space wasted on tables	Readahead possible
No limits on number of open files	Idempotency easier
No problem if a client crashes	File locking possible

Places to store files



- There are four potential places to store files:
  - The server's disk
  - The server's main memory
  - The client's disk
  - The client's main memory
- The most straightforward place to store all files is on the server's disk. There is plenty of space there and the files are then accessible to all clients. Furthermore, with only one copy of each file, no consistency problems arise.
- The problem with using the server's disk is performance. Before a client can read a file, the file must first be transferred from the server's disk to the server's main memory, and then again over the network to the client's main memory. Both transfers take time.
- A considerable performance gain can be achieved by **caching** the most recently used files in the **server's main memory**.

- To eliminate network traffic, put the cache in **client's main memory**.
- There are **three** options to where to put it:
-

1.Put the cache within each process

- **Advantage**: extremely low overhead
- **Disadvantage**: only effective if individual processes open and close files repeatedly.
- **A database manager process** might fit this, but in the usual program development environment, most processes only read each file once, so caching within the library wins nothing.

Put the cache in the kernel

- **Advantage**: the cache survives the process more than compensates. E.g. suppose a two-pass compiler runs as two processes. Pass one writes an intermediate file read by pass two. After the pass one process terminates, the intermediate file will probably be in the cache, so no server calls will have to be made when the pass two process reads it in.
- **Disadvantage**: a kernel call is needed in all cases.

Put the cache in a separate user-level cache manager process

- **Advantage**: it keeps the kernel free of file system code, is easier to program because it is completely isolated, and is more flexible.
- **Disadvantage**: when the kernel manages the cache, it can dynamically decide how much memory to reserve for programs and how much for the cache. With a user-level cache manager running on a machine with virtual memory, it is conceivable that the kernel could decide to page out some or all of the cache to a disk, so that a so-called “cache hit” requires one or more pages to be brought in. This defeats the idea of client caching completely. However, if it is possible for the cache manager to allocate and lock in memory some number of pages, that helps.

- In summary, if the network is slow and RPCs are fast, it is good to use cache. Otherwise, there is no gain using cache.

Cache Consistency

- Solution 1: Write through
- Solution 2: Delayed write
- Solution 3: Write-on-Close
- Solution 4: Centralized control algorithm
- Solution 5: Use immutable files

Write through

- When a cache entry (file or block) is modified, the new value is kept in the cache, but is also immediately sent to the server.
- Problem (1): a process A reads file *f* and then terminates, but *f* is kept in the cache of the machine. A process B modifies the same file and write through to the server. A new process in A wants to read *f* and gets the old version.
- Solution: the cache manager should check with server whether the file in the cache is an up-to-date one or not.
- Problem (2): it helps on reads, the network traffic for writes is the same as if there were no caching at all.

Delayed write

- Instead of going to the server the instant the write is done, the client just makes a note that a file has been updated. Once every 30 sec or so, all the file updates are gathered together and sent to the server at once.

Write-on-Close

- Only write a file back to the server after it has been closed.

Centralized control algorithm

- When a file is opened, the machine opening it sends a message to the file server to announce this fact. The file server keeps track of who has which file open, and whether it is open for reading, writing, or both.
- If for reading, Ok. If for writing, all other access must be prevented until the file is closed. It is UNIX semantics, but not robust and scales poorly (when a client tries to open an already opened file, the request can either be denied or queued).

Use immutable files

- Cache it on machine A. Without worrying about that machine B will change it.

File Replication

- **Why file replication?**

1. To increase reliability by having independent backups of each file. If one server goes down, or is even lost permanently, no data are lost.
2. To allow file access to occur even if one file server is down. A server crash should not bring the entire system down until the server can be rebooted.
3. To split the workload over multiple servers. As the system grows in size, having all the files on one server can become a performance bottleneck. By having files replicated on two or more servers, the least heavily loaded one can be used

There are **three** ways replication can be done.

- **Explicit file replication**

  This is for the programmer to control the entire process.

- **Lazy replication**

  Only one copy of each file is created, on some server. Later, the server itself makes replicas on other servers automatically, without the programmer's knowledge.

- **Group communication**

  All WRITE system calls are simultaneously transmitted to all the servers at once, so extra copies are made at the same time the original is made.

Update Protocols

- Centralized solution
- Distributed solutions

Centralized solution

- **A centralized solution** involves the designation of one file server as the primary server for a set of files. All requests to update data are handled through this primary server. When the primary server is down, updates may not take place but the files are still available via the secondary servers for reading.
- Disadvantage: if the primary is down, no updates can be performed.

Distributed solutions

- **The first solution** utilizes group communication. Whenever a given participant changes the contents of a file, it communicates the write commands to all participants.
- **The second solution** involves voting and the association of version numbers. A client requests permission to modify a file from the various servers. Permission is achieved by a majority of the servers agreeing on the latest version along with the stipulation that no server has communicated the existence of any version number that is higher.

- **Voting** (proposed by Gifford)

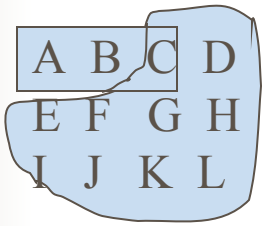
  The basic idea is to require clients to request and acquire the permission of multiple servers before either reading or writing a replicated file.

- If client wants to read, acquire **N/2+1** (majority) servers.
- If client wants to write, acquire **N/2+1** servers.

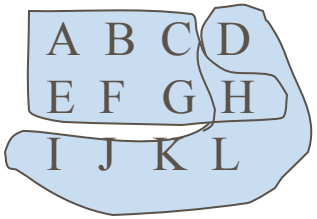
A B C D
E F G H
I J K L

- Gifford's scheme is more general.

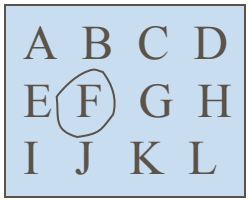
Nr+Nw>N



Nr =3, Nw =10



Nr = 7, Nw = 6



Nr = 1, Nw = 12

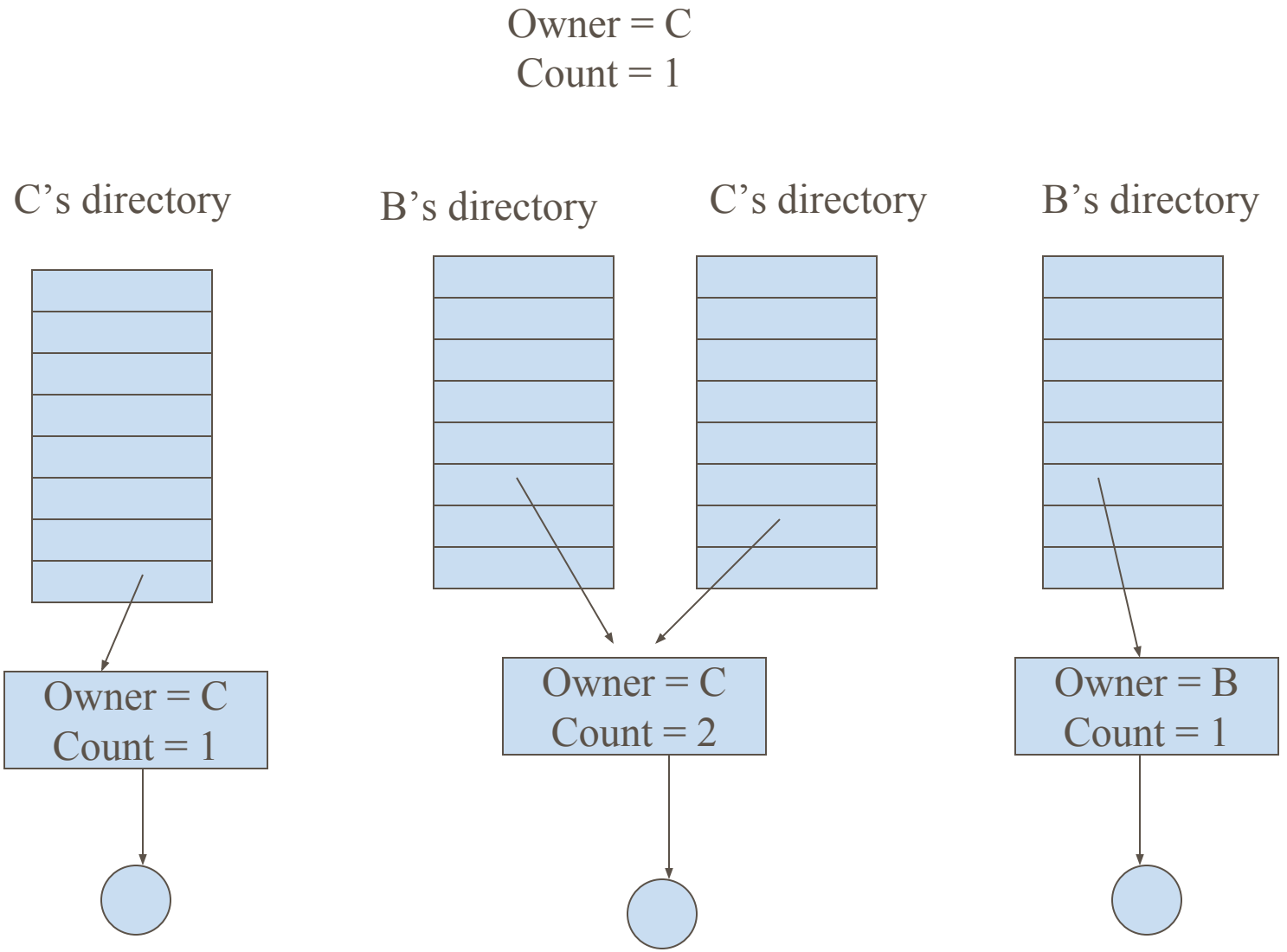
Directory Structures

- **Hierarchical directory structure**: allows directories and subdirectories. A subdirectory may only have one parent directory.

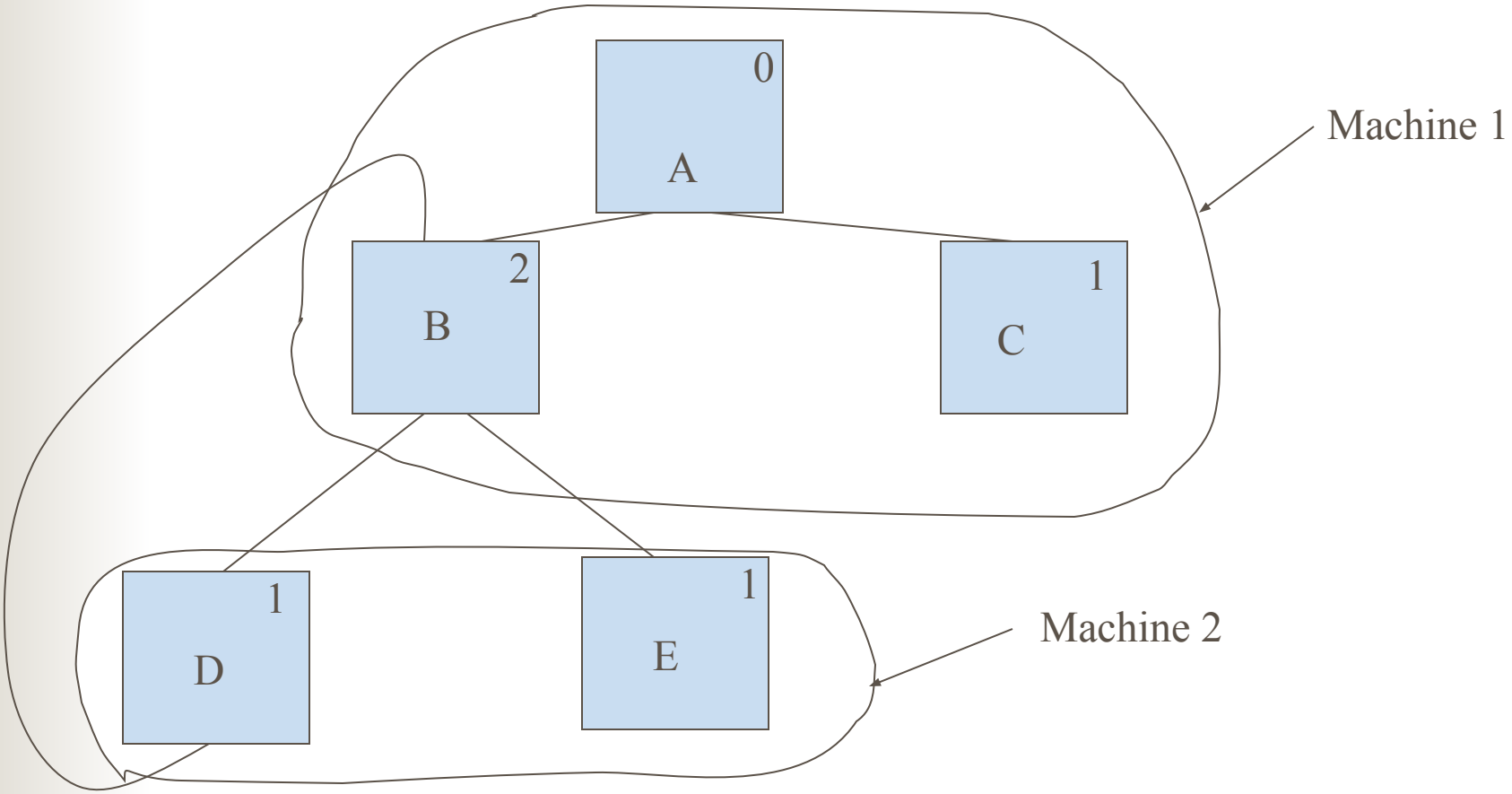
  This allows users to organize their files easily but makes it difficult for multiple users to share files.

- **Acyclic directory structure**: allows an acyclic graph structure which lets a directory to have multiple parent directories.

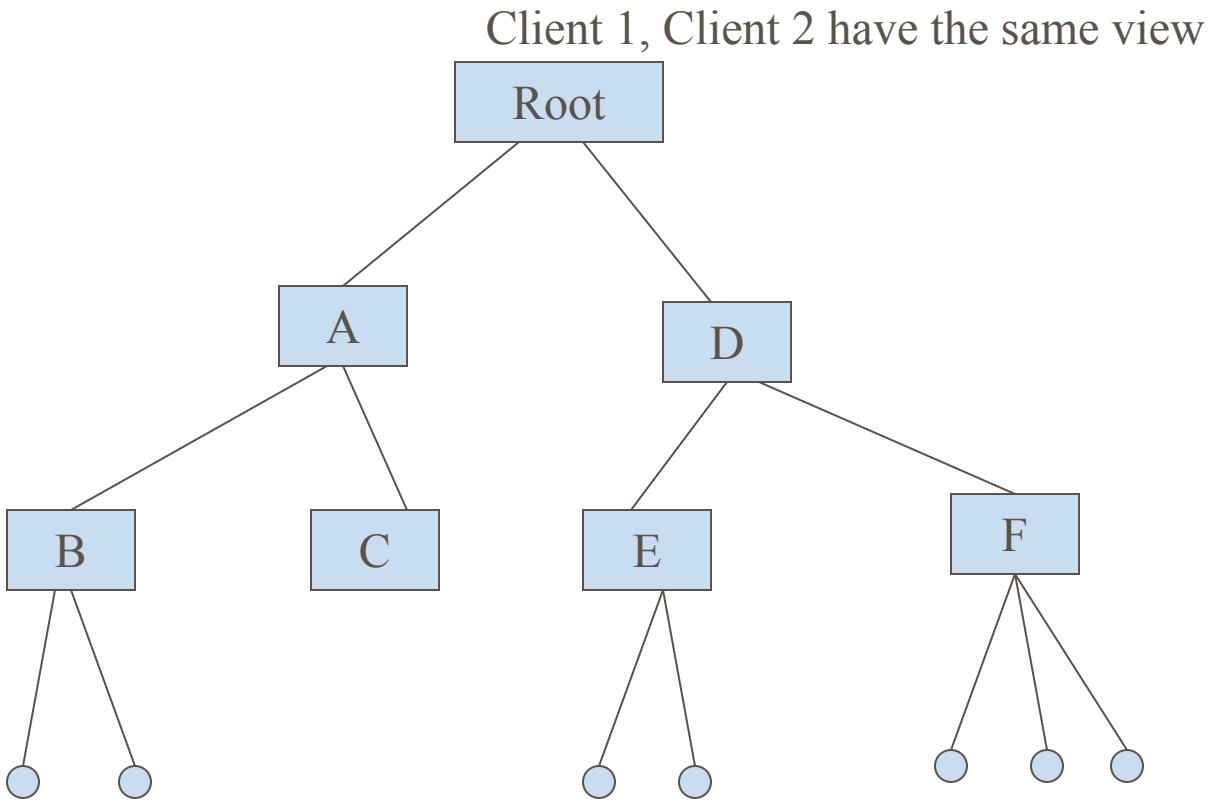
  This provides for easy file sharing but complicates directory management. E.g. **Unix** system


Owner = C
Count = 1
C's directory
B's directory
C's directory
B's directory
Owner = C
Count = 1
Owner = C
Count = 2
Owner = B
Count = 1

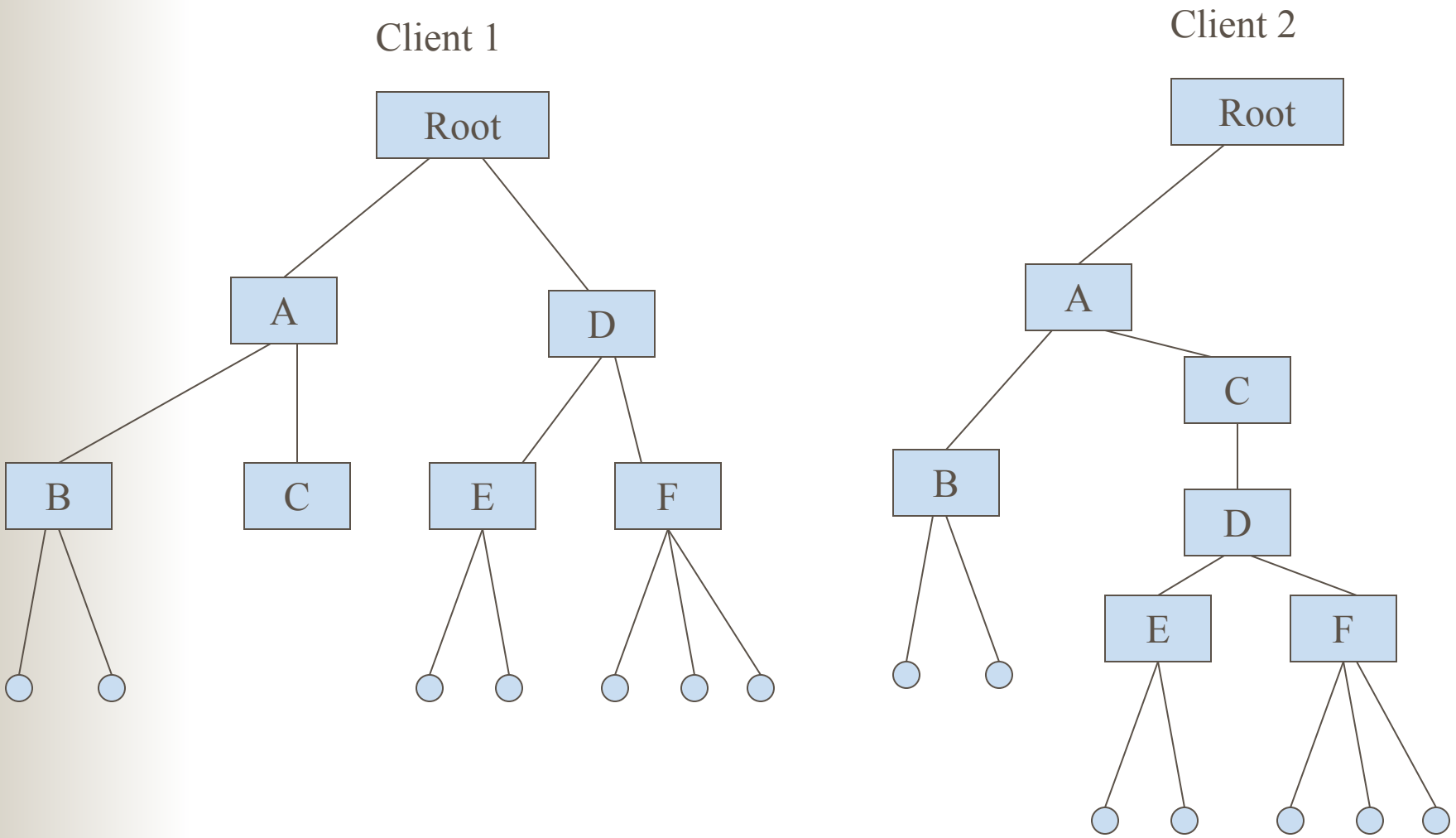
A



All clients have the same view



Different clients have different view



Directory Operations

- Create a directory
- Delete a directory
- Rename a directory
- List a directory's contents
- Manage a directory's access permissions
- Changing a directory's access permissions
- Move a directory within the overall directory structure
- Traverse the entire directory structure

Trends in distributed file systems

- **New Hardware**
  - Memory price is cheaper and cheaper
  - Optical disk
  - Very fast fiber optic networks
- **Scalability**
- **Wide area networking**
- **Mobile users**
- **Fault Tolerance**
- **Multimedia**